

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 03 -

ENTREGA CONTÍNUA PARA KUBERNETES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	12
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EMANDA

O QUE VEM POR AÍ?

Bem-vindo e bem-vinda de volta à nossa jornada! É hora de aplicar os conceitos de CI/CD diretamente no coração das arquiteturas modernas: o Kubernetes. Nesta aula, você irá desvendar as estratégias de build, push e deploy para entregar software de forma ágil e segura em ambientes de contêineres.



HANDS ON

A teoria ganha vida nas próximas videoaulas. Você construirá um pipeline completo, automatizando o build e o push de uma imagem Docker para, em seguida, fazer o deploy no Kubernetes. Exploraremos na prática ferramentas para gerenciar manifestos e aplicaremos estratégias avançadas de deploy, como Blue/Green e Canary. Veja como esses conceitos se encaixam em um fluxo de trabalho real para garantir lançamentos mais seguros.



SAIBA MAIS

Uma Visão Estratégica da Entrega Contínua para Kubernetes

A transição da entrega de software de ambientes tradicionais para o ecossistema dinâmico do Kubernetes é um salto de paradigma. A filosofia DevOps incentiva a entrega de pequenas e frequentes mudanças de código, resultando em centenas de deploys por dia em uma infraestrutura massiva. A natureza escalável dos contêineres, orquestrada pelo Kubernetes, torna os processos de entrega manual um gargalo insustentável.

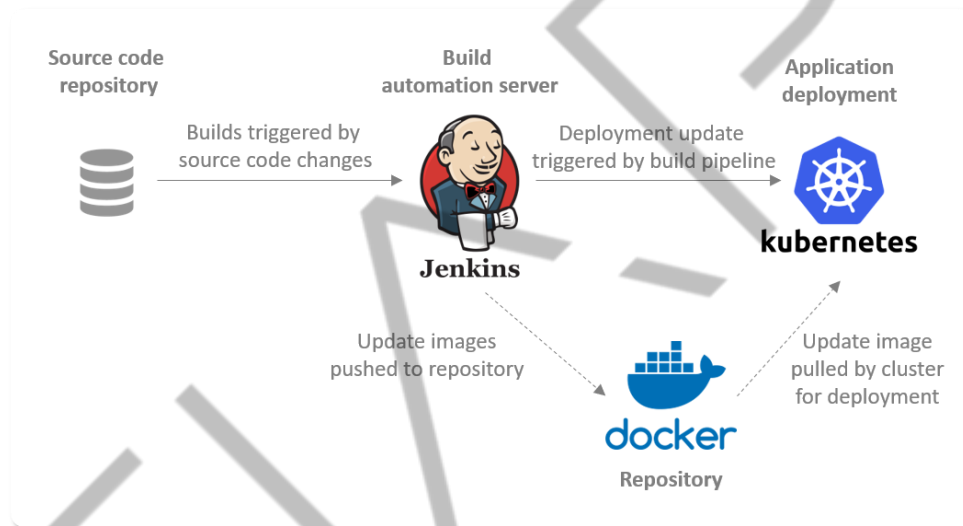


Figura 1 - CI/CD para Kubernetes
Fonte: [Média](#) (2025)

A automação do pipeline de Entrega Contínua (CD) emerge, portanto, não como uma opção, mas como um imperativo estratégico. Um pipeline de CD para Kubernetes vai além da simples compilação e execução de testes. Ele empacota o código em imagens de contêiner, as envia para um registro e, finalmente, orquestra o deploy de forma segura. Essa necessidade de automação ponta a ponta é a razão pela qual as práticas de Pipeline as Code são tão cruciais, permitindo que a definição do fluxo de entrega evolua junto com o próprio código.

Gerenciando a Complexidade dos Manifestos: Helm e Kustomize em Detalhe

Quando uma aplicação cresce, a gestão manual de arquivos de manifesto YAML pode se tornar assustadora. Nesse cenário, o mercado se divide entre duas abordagens principais: o modelo de templating do Helm e o modelo de overlay do Kustomize.

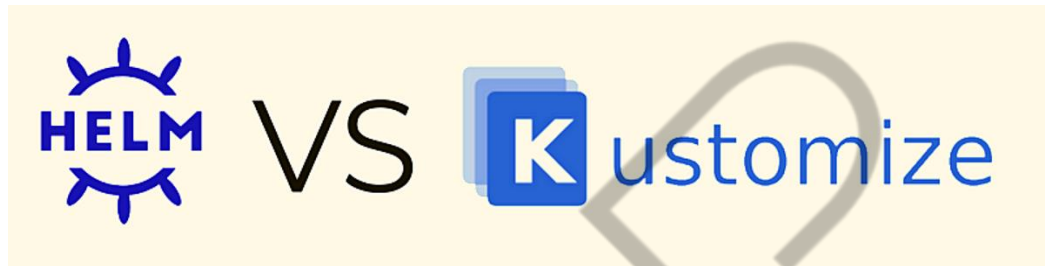


Figura 2 - Helm vs. Kustomize
Fonte: [Medium](#) (2025)

O Helm é o "gerenciador de pacotes" para Kubernetes, com uma filosofia baseada em charts que contêm templates de arquivos YAML. Esses templates usam variáveis que podem ser substituídas por valores específicos de cada ambiente, permitindo a criação de pacotes reutilizáveis e versionados. O Helm também oferece funcionalidades de gerenciamento de releases e rollbacks simplificados. Sua natureza como ferramenta externa ao Kubernetes pode adicionar uma camada de complexidade e segurança.

Em contraste, o Kustomize adota uma abordagem de "motor de overlay", trabalhando diretamente com manifestos YAML brutos. Você define uma base de manifestos e, para cada ambiente, cria um arquivo de customização que especifica pequenas mudanças (patches) a serem aplicadas. A grande vantagem do Kustomize é sua simplicidade e a sua integração nativa ao kubectl, o que o torna uma escolha natural para fluxos de trabalho declarativos e GitOps.

A escolha entre as duas ferramentas não é uma questão de qual é "melhor", mas de qual se adequa mais à sua filosofia de trabalho. O Helm é ideal para quem precisa de um sistema robusto de empacotamento, enquanto o Kustomize é perfeito para equipes que precisam customizar manifestos de forma simples e declarativa. É comum, inclusive, usar as duas ferramentas em combinação.

Característica	Helm	Kustomize
Abordagem	Motor de Templating	Motor de Overlay/Patch
Tipo de Ferramenta	Gerenciador de Pacotes	Ferramenta de Customização
Vantagens	Reutilização de pacotes, gerenciamento de releases, versionamento, rollback fácil.	Simplicidade, integração nativa ao kubectl, evita lógica em manifestos.
Desvantagens	Introduz nova camada de complexidade e segurança, sintaxe de templating.	Não gerencia releases nem dependências de forma nativa.
Integração com kubectl	Não nativa; requer instalação separada.	Nativa; funcionalidade embutida no comando apply.

Tabela 1 – Comparativo Helm vs. Kustomize
Fonte: elaborado pelo autor (2025)

Anatomia de um Deploy Seguro em Kubernetes: Blue/Green e Canary

O estágio de deploy em um pipeline de Entrega Contínua para Kubernetes é, sem dúvida, o ponto mais crítico. Para mitigar riscos, o mercado adota estratégias avançadas, sendo as mais proeminentes o Blue/Green e o Canary.

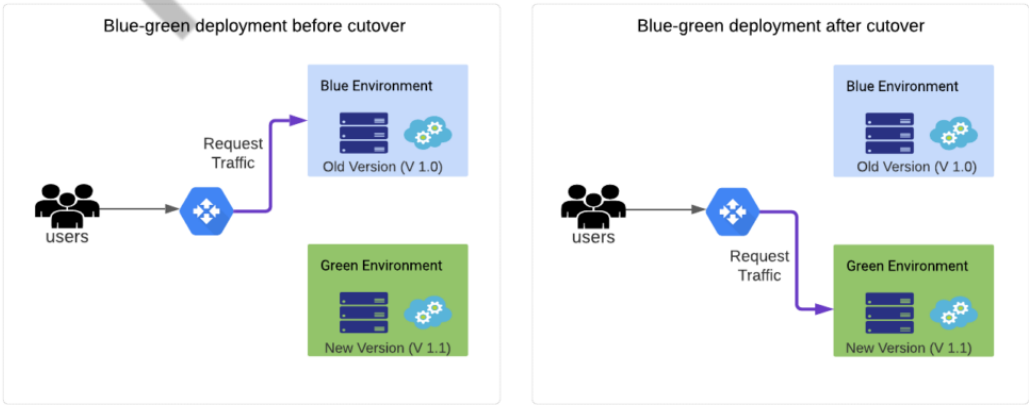


Figura 3 - Blue/Green Deployment

Fonte: <https://www.w6d.io/blog/wp-content/uploads/sites/2/2021/08/bluegreen-1024x437.png> (2025)

A estratégia Blue/Green (azul/verde) visa downtime zero e capacidade de rollback instantâneo. A filosofia é simples: mantenha dois ambientes idênticos em produção, um chamado "Blue" (a versão atual e ativa) e o outro, "Green" (a nova versão). O tráfego é instantaneamente redirecionado para o ambiente "Green" após a validação da nova versão. A grande vantagem é a segurança e a rapidez do rollback, enquanto a desvantagem é o custo, já que é preciso duplicar a infraestrutura.

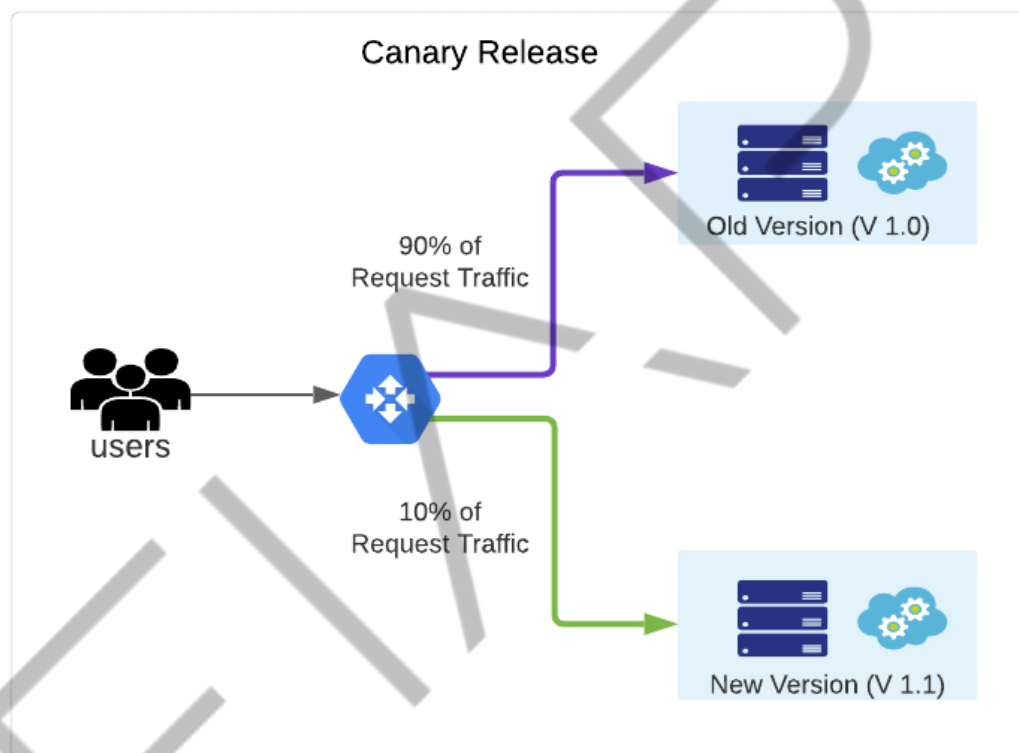


Figura 4 – Canary Deployment

Fonte: <https://www.w6d.io/blog/wp-content/uploads/sites/2/2021/08/canary.png> (2025)

Em oposição, o Canary deployment (deploy canário) é uma estratégia que minimiza o risco ao expor a nova versão a um pequeno subconjunto de usuários. Se não houver problemas, o tráfego para a nova versão é gradualmente aumentado. Essa abordagem é ideal para testar novas funcionalidades em um ambiente de produção com risco mínimo e para coletar feedback em tempo real de um subconjunto de usuários. A implementação de um canary deployment em Kubernetes não é nativa e exige ferramentas adicionais.

A escolha entre as duas estratégias se resume a um trade-off fundamental: Blue/Green foca na velocidade e na garantia de downtime zero, enquanto Canary prioriza o controle gradual de risco e a coleta de feedback.

Característica	Blue/Green	Canary
Estratégia	Switchover total entre dois ambientes.	Lançamento gradual e incremental.
Filosofia de Lançamento	Tudo ou nada.	Faseado, com controle de risco.
Vantagens	Downtime zero, rollback instantâneo.	Mitigação de risco, coleta de feedback em tempo real.
Desvantagens	Exige duplicação de infraestrutura (mais caro).	Processo mais lento, requer ferramentas de gestão de tráfego.
Casos de Uso	Atualizações críticas, alta disponibilidade.	Lançamento de novas funcionalidades, testes em produção.

Tabela 2 – Comparativo Blue/Green vs. Canary

Fonte: elaborado pelo autor (2025)

A Ascensão do GitOps: O Futuro da Entrega Contínua e a Engenharia de Plataformas

Uma das tendências mais transformadoras é a ascensão do GitOps. Essa abordagem eleva o conceito de Pipeline as Code a um novo patamar, tratando o Git como a "única fonte de verdade" para o estado desejado da infraestrutura e das aplicações em um cluster Kubernetes.

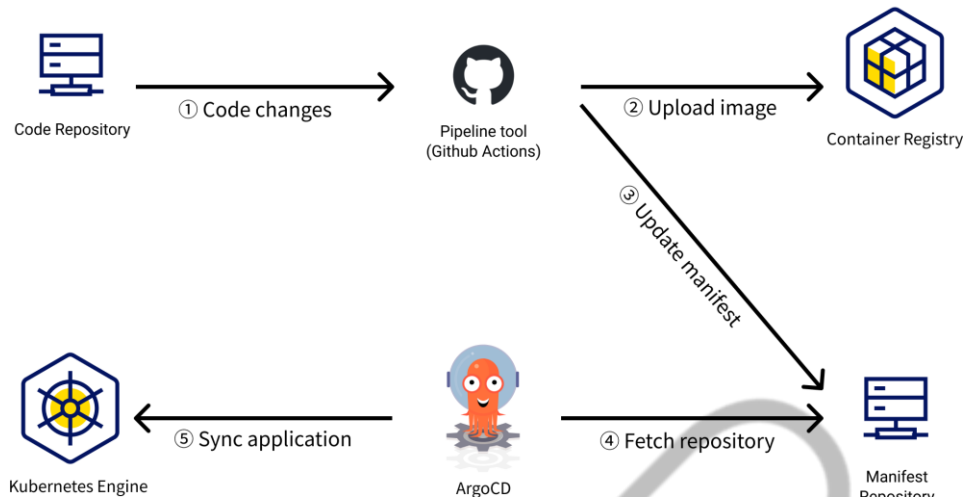


Figura 5 – GitOps Workflow

Fonte: [https://objectstorage.kr-central-](https://objectstorage.kr-central-2.kakaocloud.com/v1/c11fcba415bd4314b595db954e4d4422/public/tutorial/gitops-pipeline/GitOps.png)[2.kakaocloud.com/v1/c11fcba415bd4314b595db954e4d4422/public/tutorial/gitops-pipeline/GitOps.png](https://objectstorage.kr-central-2.kakaocloud.com/v1/c11fcba415bd4314b595db954e4d4422/public/tutorial/gitops-pipeline/GitOps.png)

(2025)

Diferente de um pipeline tradicional, que opera em um modelo push-based, o GitOps inverte essa lógica. Qualquer mudança é feita através de um pull request no repositório Git. Uma vez que a mudança é aprovada e mergeada, um operador, que está dentro do cluster, detecta a alteração e, de forma automática, pulla o novo estado e o reconcilia com o estado real do cluster. Essa abordagem elimina a necessidade de credenciais de acesso ao cluster em ferramentas de CI externas, aumentando a segurança e garantindo que o estado do cluster nunca se desvie do que está declarado no repositório.

O GitOps é a materialização do Infrastructure as Code e o alicerce para uma das tendências mais importantes do mercado: a Engenharia de Plataformas. Essa nova disciplina foca na construção de plataformas internas de autoatendimento (self-service), que padronizam e automatizam o ciclo de vida do software, permitindo que pessoas desenvolvedoras foquem na entrega de valor.

Ferramentas Essenciais no Cenário GitOps: Argo CD e Flux

Para a implementação do GitOps, o mercado tem duas ferramentas de referência: o Argo CD e o Flux.

O Argo CD é uma plataforma completa de Entrega Contínua, projetada com uma arquitetura end-to-end. Sua principal característica é a interface gráfica (web UI) rica e intuitiva, que simplifica a gestão e o monitoramento de múltiplos clusters de forma visual. Ele é conhecido por sua facilidade de uso e por ser uma ótima porta de entrada para equipes que estão migrando de modelos tradicionais para o GitOps.

Já o Flux adota uma arquitetura de "toolkit", sendo mais modular e focado na linha de comando (CLI). Em vez de ser uma aplicação monolítica, o Flux é uma coleção de controladores Kubernetes que se integram de forma mais nativa. Essa modularidade permite que as equipes customizem o fluxo de trabalho de GitOps de forma mais flexível.

A escolha entre as duas ferramentas se resume a uma decisão entre a conveniência de uma plataforma completa (Argo CD) e a flexibilidade de um toolkit modular (Flux).

Característica	Argo CD	Flux
Arquitetura	Aplicação end-to-end completa.	Toolkit modular de controladores.
Facilidade de Uso	Mais simples; ótima para iniciantes.	Curva de aprendizado um pouco mais íngreme.
UI Integrada	Sim, UI completa e intuitiva.	Não nativa; requer instalação separada.
Casos de Uso	Gerenciamento de múltiplos clusters, ambientes empresariais.	Ambientes com restrições de recursos, equipes que preferem CLI.
Integração	Forte com o ecossistema Argo.	Forte com o ecossistema Kubernetes e outros projetos da CNCF.

Tabela 3 – Comparativo Argo CD vs. Flux
Fonte: elaborado pelo autor (2025)

MERCADO, CASES E TENDÊNCIAS

O mercado de soluções para Kubernetes e Entrega Contínua está em um momento de crescimento explosivo, impulsionado pela adoção massiva de arquiteturas nativas da nuvem. Esses dados mostram que a automação da entrega de software não é mais uma vantagem, mas um pré-requisito para o sucesso.

O futuro dos pipelines de CI/CD será moldado por tendências emergentes. A Inteligência Artificial (IA) já está sendo utilizada para otimizar fluxos de trabalho e prever falhas de build. Outra tendência de destaque é a ascensão do GitOps, com a maioria das empresas cloud-native já integrando essa abordagem em seus pipelines. Isso mostra que a automação declarativa se tornou o padrão.

A culminação de todas essas tendências é a Engenharia de Plataformas, que busca construir plataformas internas de autoatendimento (self-service) para institucionalizar as melhores práticas de entrega de software, liberando os(as) desenvolvedores(as) da complexidade da infraestrutura para que possam focar na entrega de valor.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, a sua compreensão sobre Entrega Contínua para Kubernetes foi expandida. Você viu a natureza estratégica do processo build -> push -> deploy e a necessidade de automação ponta a ponta.

Exploramos as duas abordagens mais influentes para a gestão de manifestos, Helm e Kustomize e desvendamos a diferença filosófica entre templating e overlay. Nos aprofundamos nas estratégias de deploy avançadas, Blue/Green e Canary e discutimos como a escolha entre elas é uma decisão que equilibra segurança e objetivos de negócio.

Por fim, fomos além do básico e mergulhamos no GitOps, o modelo operacional do futuro, e analisamos as duas ferramentas que o materializam, Argo CD e Flux.

REFERÊNCIAS

AMAZON AWS. **Use cases and comparison between Argo CD and Flux for GitOps.** 2025. Disponível em: <https://docs.aws.amazon.com/prescriptive-guidance/latest/eks-gitops-tools/use-cases.html>. Acesso em: 07 out. 2025.

BURNS, B.; BRENDAN, O.; BEDA, J. **Kubernetes Up & Running: Dive into the Future of Infrastructure.** O'Reilly Media, 2017.

KUNDARIYA, H. **Emerging DevOps Trends: Security, Scalability and Sustainability.** 2025. Disponível em: <https://devops.com/emerging-devops-trends-security-scalability-and-sustainability/>. Acesso em: 07 out. 2025.

OCTOPUS.COM. **Blue-Green vs Canary Deployments.** 2025. Disponível em: <https://octopus.com/devops/software-deployments/blue-green-vs-canary-deployments/>. Acesso em: 07 out. 2025.

ODAIZIE, D. **Kustomize vs. Helm – How to Use & Comparison.** 2025. Disponível em: <https://spacelift.io/blog/kustomize-vs-helm>. Acesso em: 07 out. 2025.

RED HAT. **What is GitOps and how does it relate to Kubernetes continuous delivery.** 2025. Disponível em: <https://www.redhat.com/en/topics/devops/what-is-gitops>. Acesso em: 07 out. 2025.

SAHOO, J. **Argo CD vs Flux CD: The Complete GitOps Comparison for Kubernetes.** 2025. Disponível em: <https://devtron.ai/blog/gitops-tool-selection-argo-cd-or-flux-cd/>. Acesso em: 07 out. 2025.

SUSNJARA, S. SMALLEY, I. **Kustomize versus Helm: What's the difference?** 2025. Disponível em: <https://www.ibm.com/think/insights/kustomize-vs-helm>. Acesso em: 07 out. 2025.

PALAVRAS-CHAVE

Entrega Contínua. Kubernetes. CI/CD. Docker. Automação. Helm. Kustomize. Blue/Green. Canary. GitOps. Argo CD. Flux. DevOps.

ENTRADA



POSTECH